ZeRO

Danyang Zhuo

Duke University

Some materials from “ZeRO & DeepSpeed: New system optimizations enable training models with over 100 billion parameters” and Hao Zhang’s slides

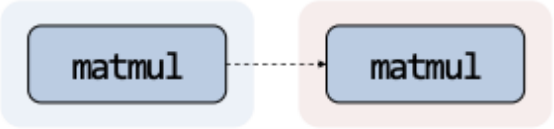
Course Project

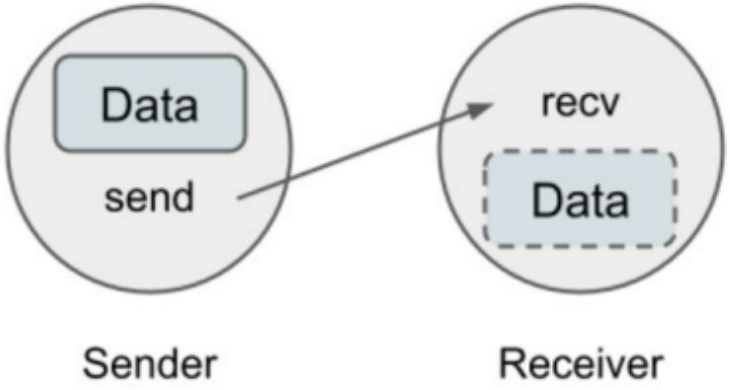
- https://courses.cs.duke.edu/spring26/compsci590.6/project.html

- Please submit via Canvas

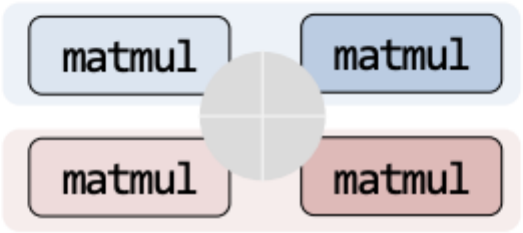
- One group only needs to submit one proposal

Terminologies: Point-to-point Communication





Terminologies: Collective Communication




```
ddp_model = DDP(Model(), device_ids=[rank])
for batch in data_loader:
    loss = train_step(ddp_model, batch)
```

Implicit allreduce here

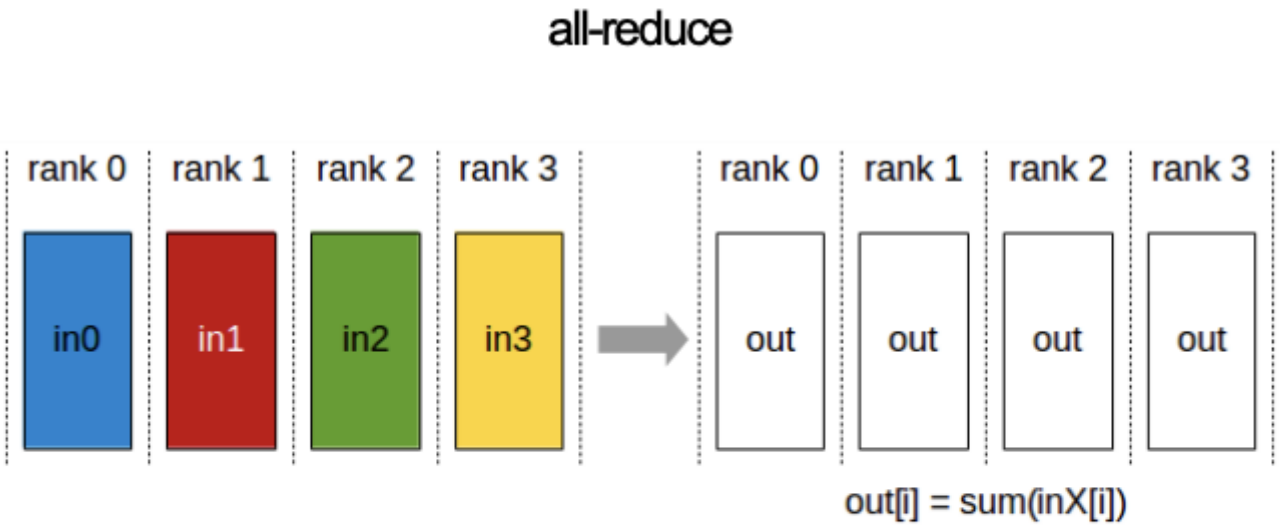
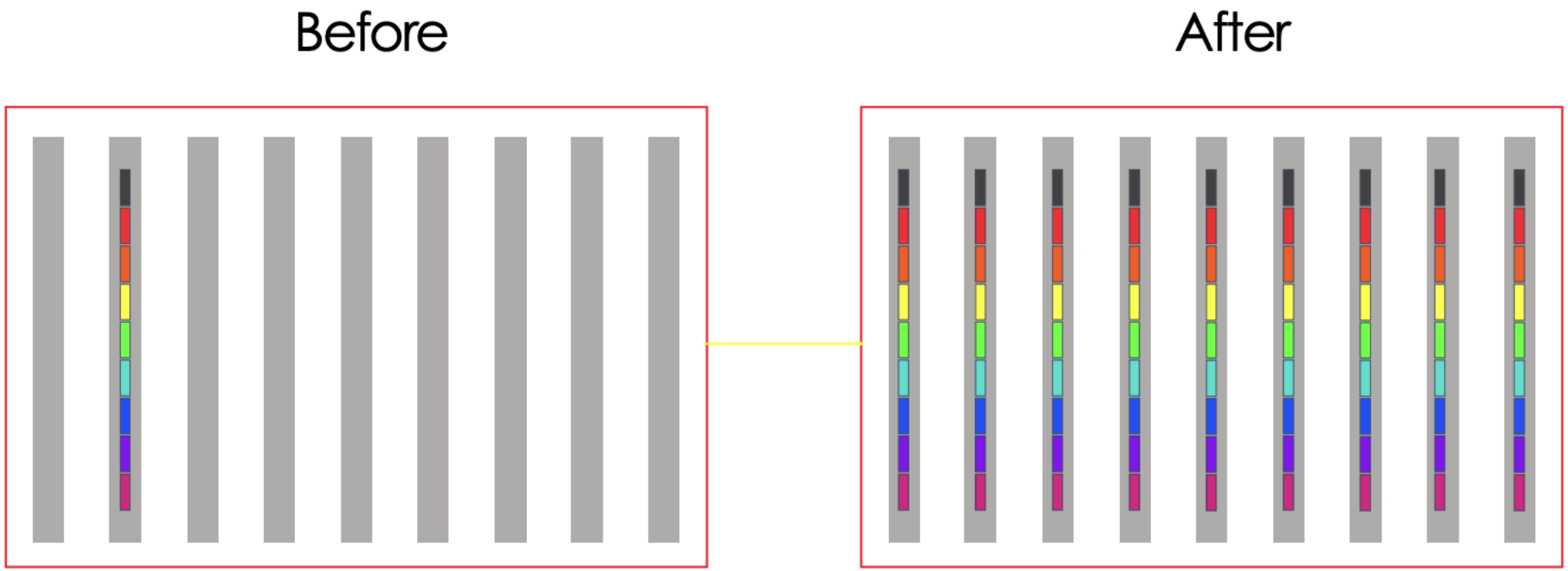


Figure from NCCL documentation

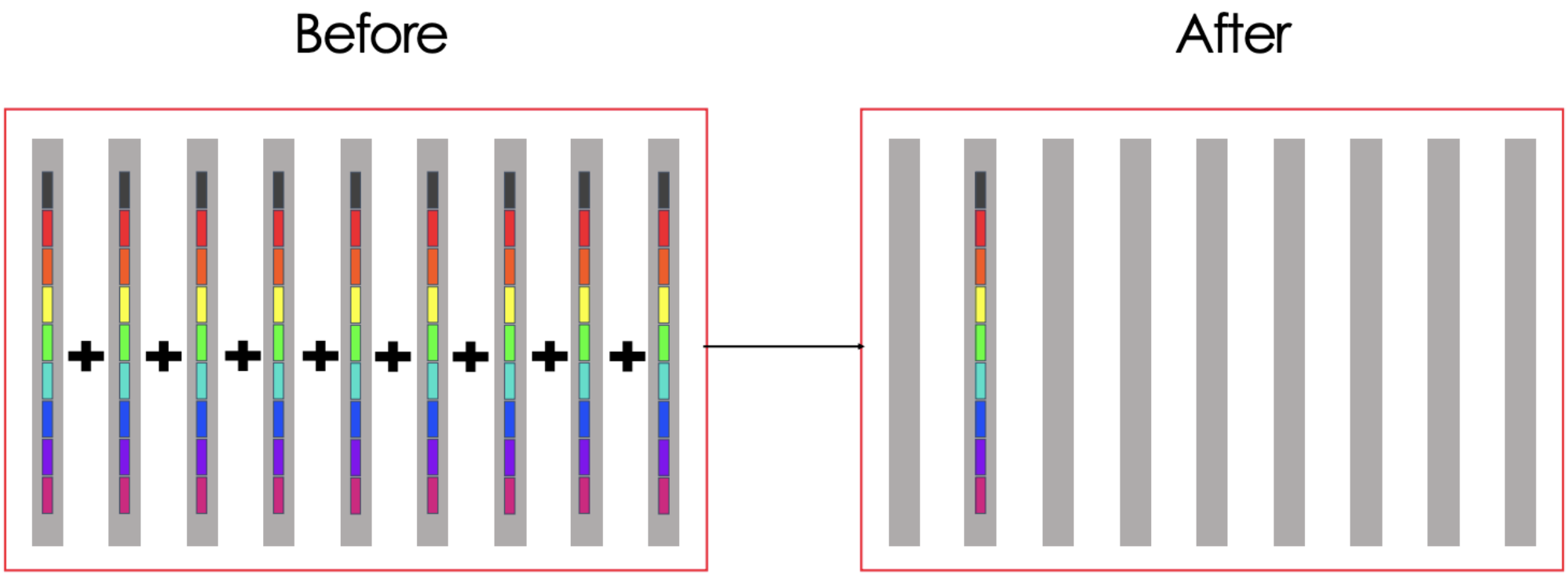
Collective Communications

- Broadcast
- Reduce(-to-one)
- Scatter
- Gather
- Allgather
- Reduce-scatter
- Allreduce

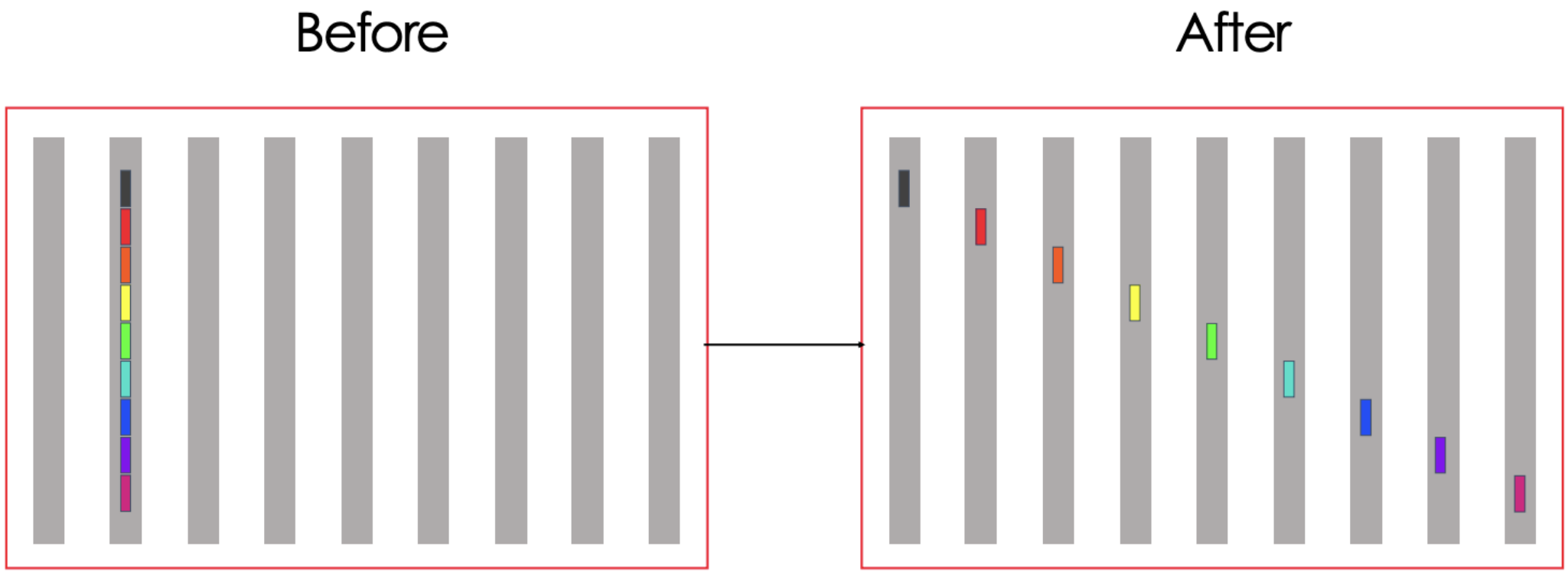
Broadcast



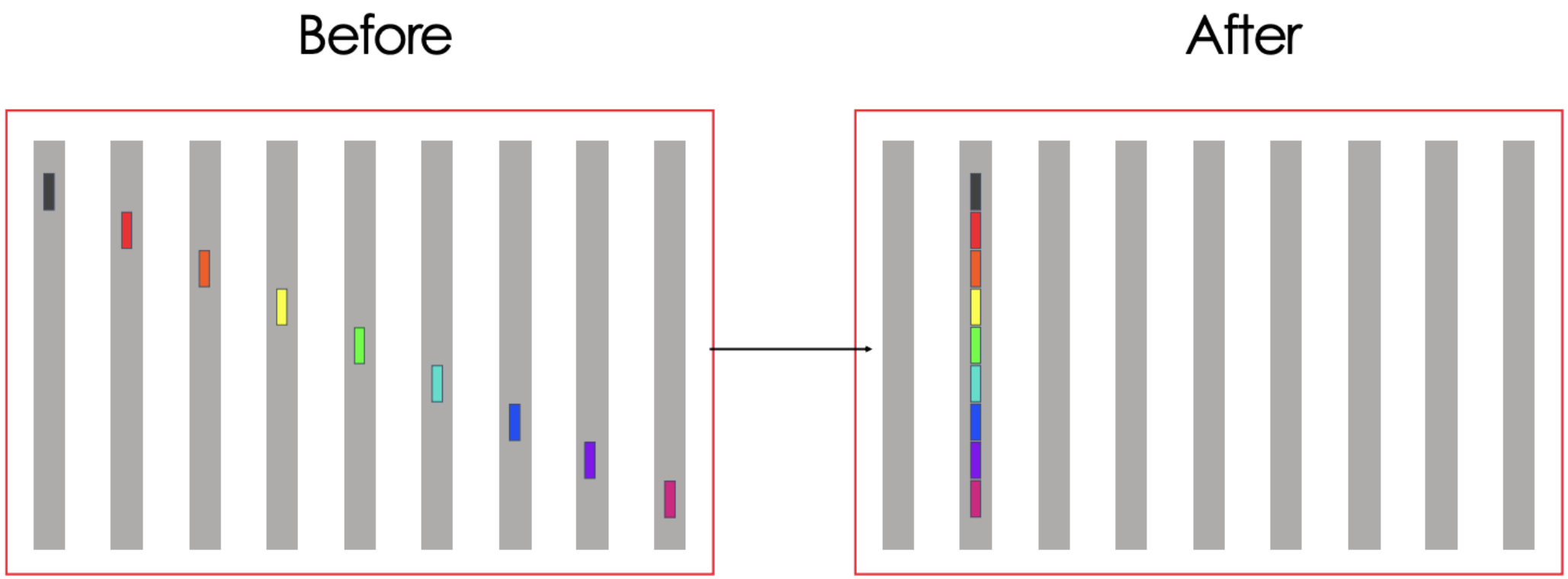
Reduce(-to-one)



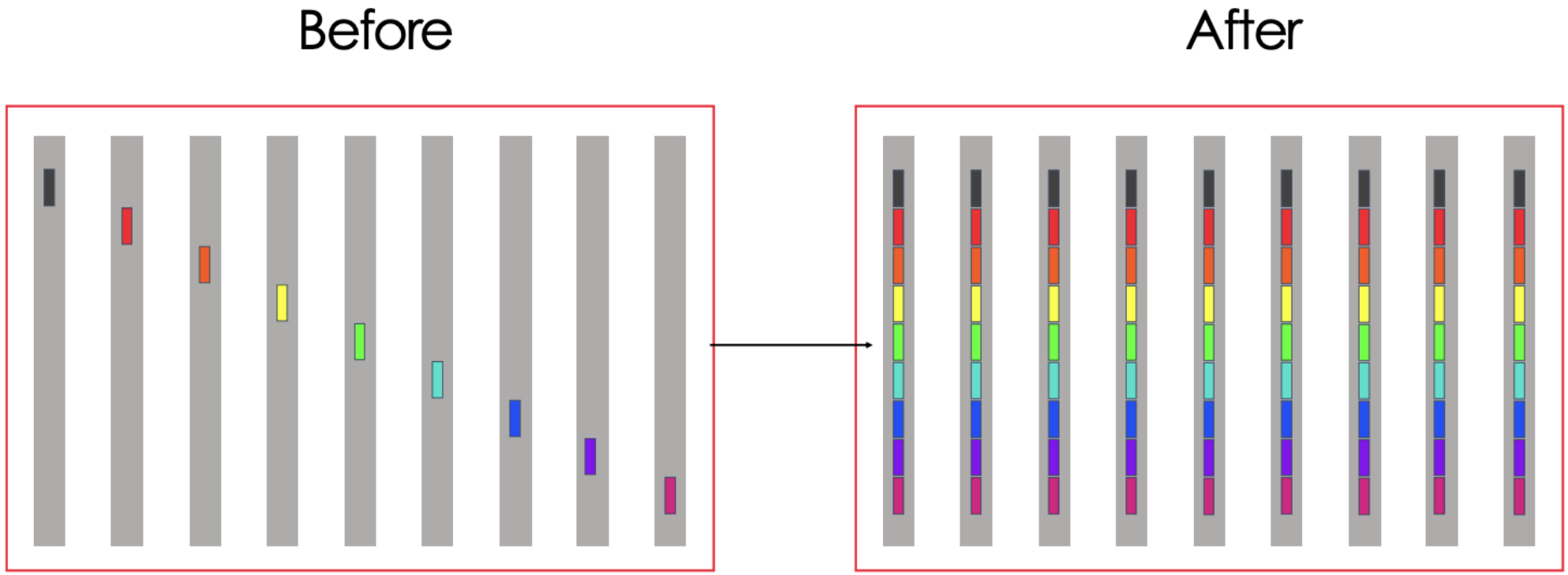
Scatter



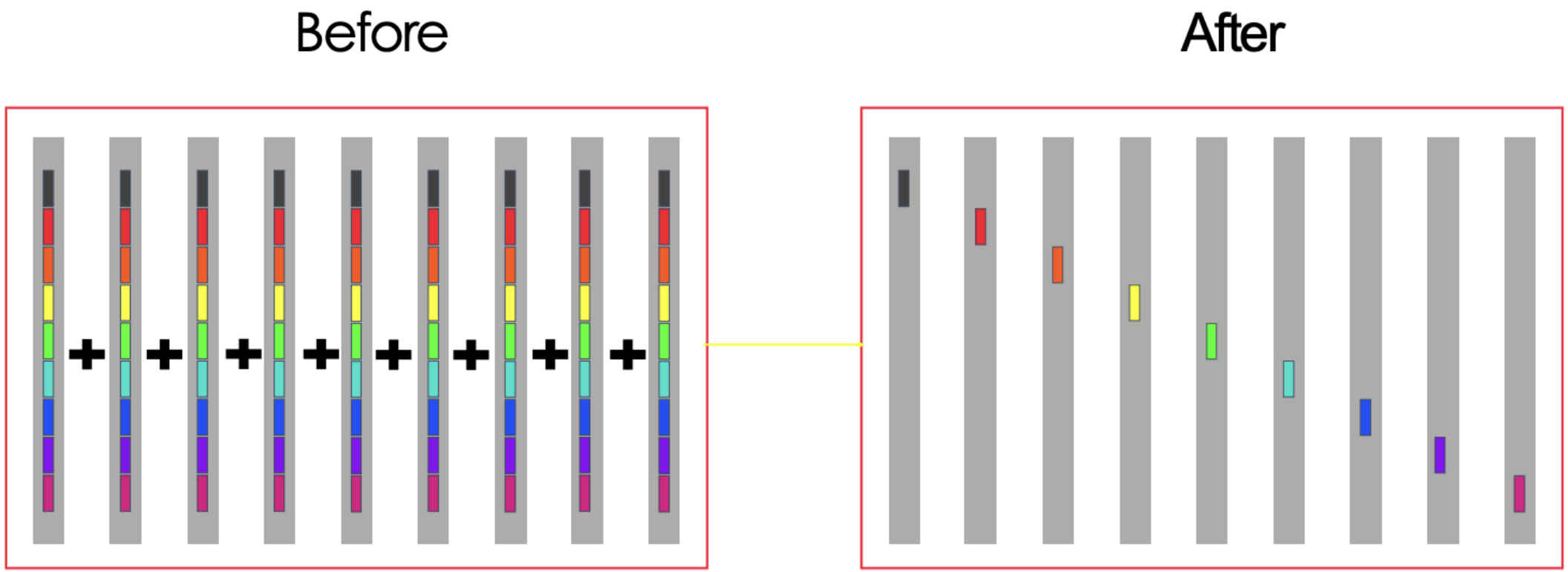
Gather



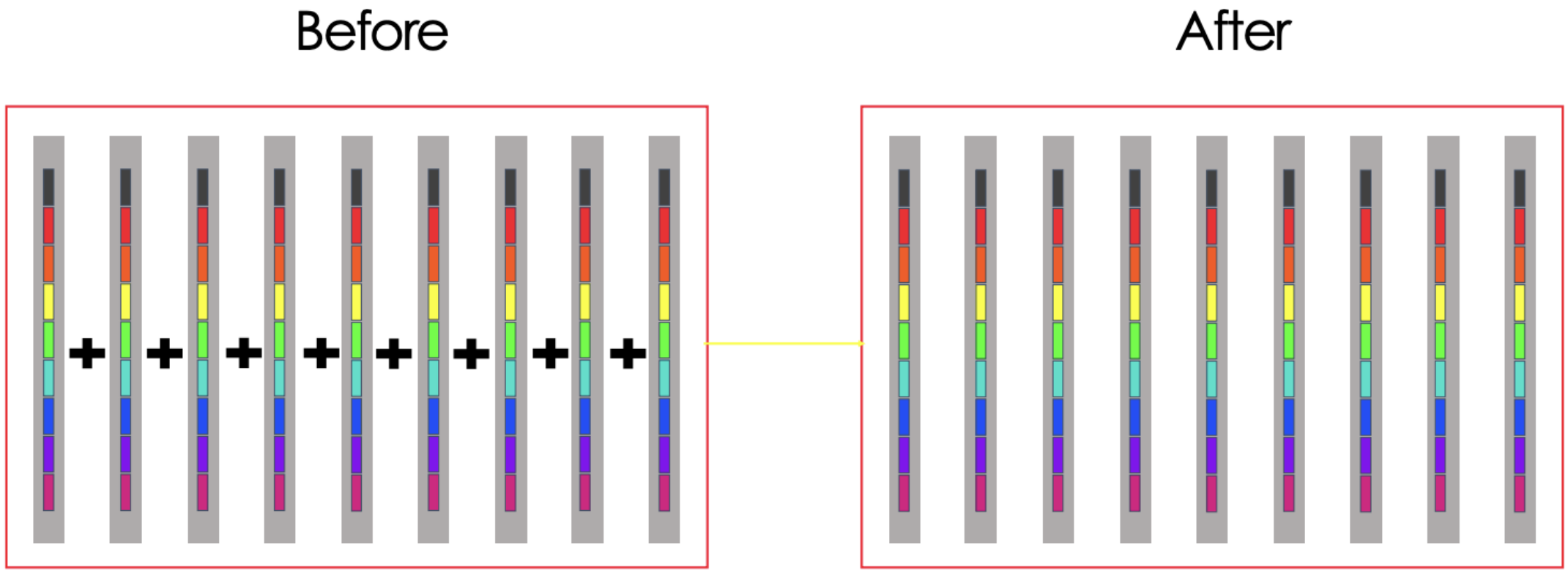
Allgather



Reduce-scatter



Allreduce



Communication Model: $\alpha\beta$ model

Communication Model: $\alpha + n\,\beta, \beta = \frac{1}{B}$

- Small Message size ($n \to 0$): α dominates, emphasize latency
- Large Message Size ($n \to +\infty$): $n\,\beta$ dominate, emphasize bandwidth utilization

Two Family of Mainstream Algorithms/Implementations

- Small message: Minimum Spanning Tree algorithm
  - Emphasize **low latency**

- Large Message: Ring algorithm
  - Emphasize **bandwidth utilization**

- There are 50+ different algorithms developed in the past 50 years by a community called "High-performance computing"
  - 2023 Turing award

Let’s think about a few collectives

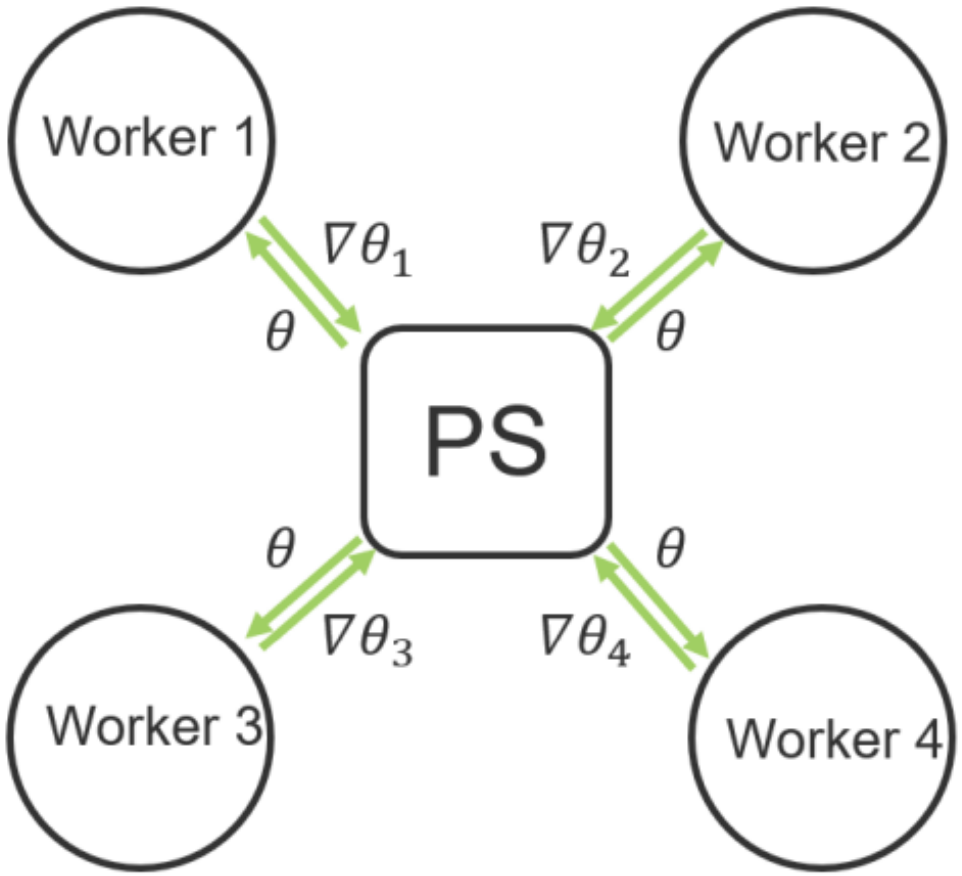
- Broadcast
  - Latency: Tree-Broadcast
  - Throughput: Scatter +Allgather
- ReduceScatter
  - Latency: Tree-Reduce + Scatter
  - Throughput: Ring-ReduceScatter
- AllGather
  - Latency: Gather + Broadcast
  - Throughput: Ring-AllGather
- AllReduce
  - Latency: Reduce + Broadcast
  - Throughput: ReduceScatter + AllGather

Data Parallelism

- Torch DDP relies on AllReduce
  - Every DP replica has a copy of the model weights

- AllReduce = Reduce + Broadcast

Parameter Server Naturally emerges



Problems:

Server bottleneck!

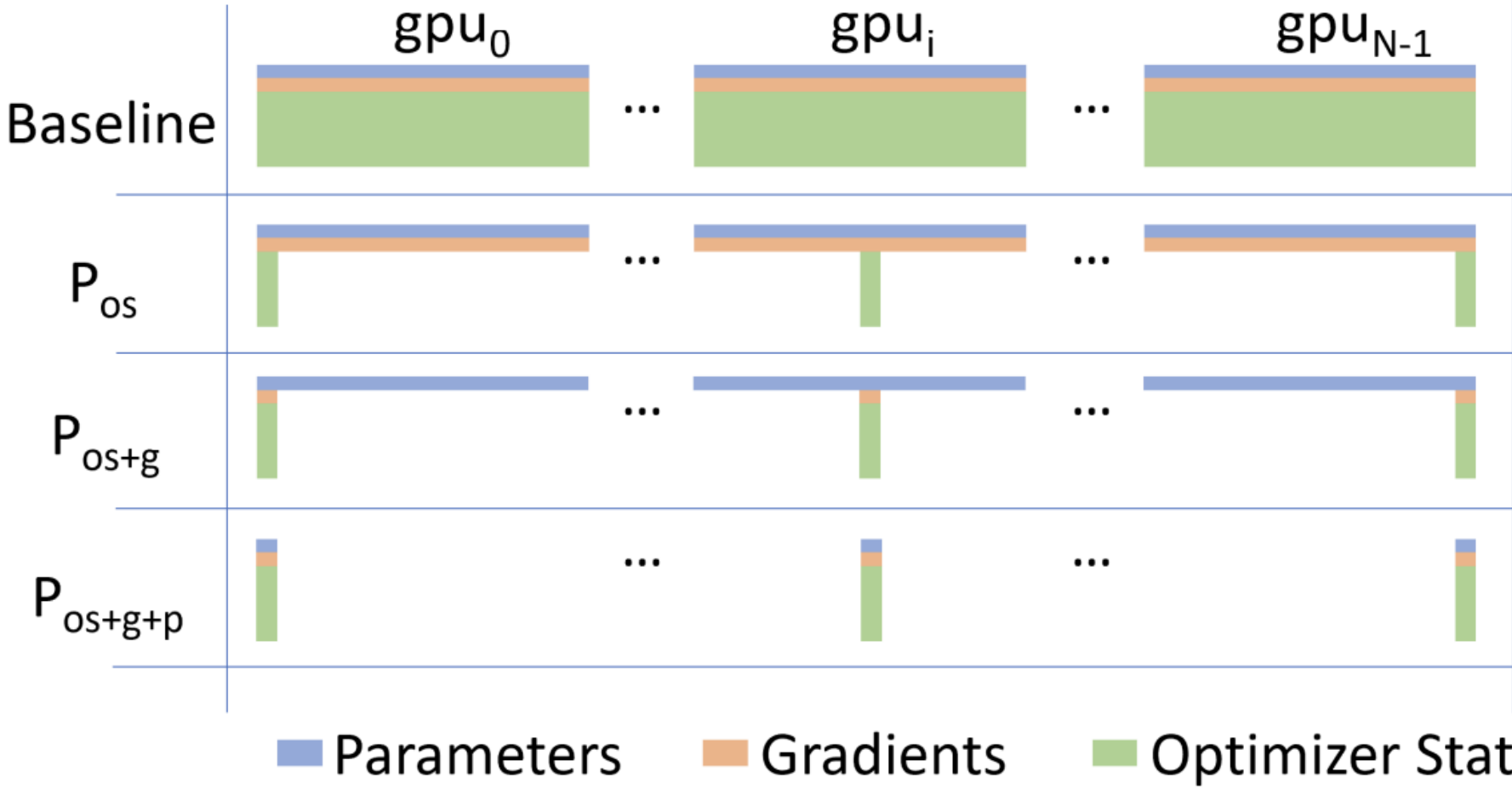
ZeRO: Memory Optimizations Toward Training Trillion Parameter Models

Samyam Rajbhandari*, Jeff Rasley*, Olatunji Ruwase, Yuxiong He
{samyamr, jerasley, olruwase, yuxhe}@microsoft.com

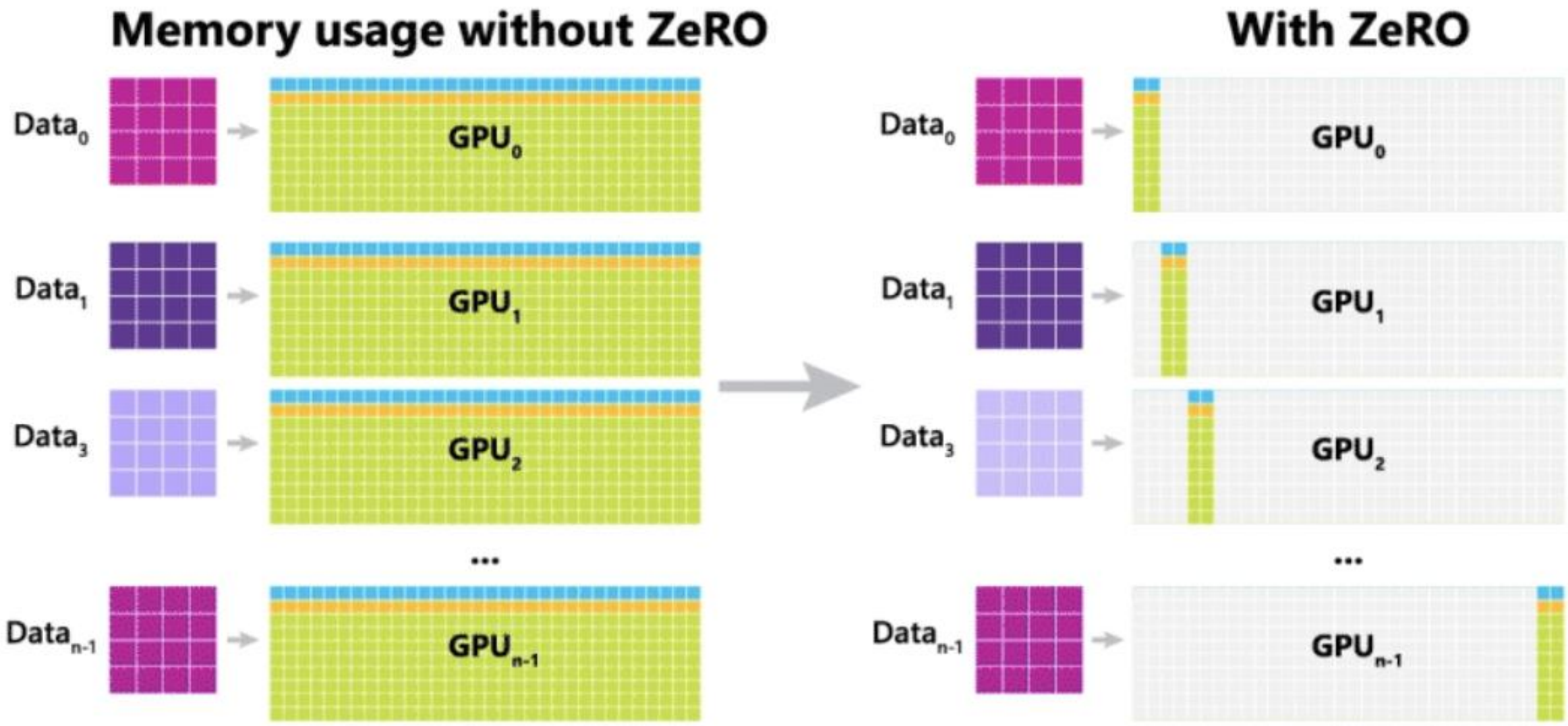
Problem

- Basic data parallelism (DP) does not reduce memory per device, and runs out of memory for models with more than 1.4B parameters on current generation of GPUs with 32 GB memory.

- MP cannot scale much further beyond these models sizes. MP splits the model vertically, partitioning the computation and parameters in each layer across multiple devices, requiring significant communication between each layer.
  - We tested a 40B parameter model using Megatron-LM across two DGX-2 nodes and observe about 5 T flops per V100 GPU (less than 5% of hardware peak).



Memory Consumed	K=12 Ψ=7.5B N_d=64
$(2+2+K)*\Psi$	120GB
$2\Psi+2\Psi+\frac{K*\Psi}{N_d}$	31.4GB
$2\Psi+\frac{(2+K)*\Psi}{N_d}$	16.6GB
$\frac{(2+2+K)*\Psi}{N_d}$	1.9GB


Memory usage without ZeRO
With ZeRO
Data0
GPU0
Data1
GPU1
Data3
GPU2
...
Datan-1
GPUn-1
Data0
GPU0
Data1
GPU1
Data3
GPU2
...
Datan-1
GPUn-1

ZeRO 4-way data parallel training

Using:

- P_{os} (Optimizer state)
- P_g (Gradient)
- P_p (Parameters)

ZeRO-DP

a) DP has better scaling efficiency than MP because MP reduces the granularity of the computation while also increasing the communication overhead. Beyond a certain point, lower computational granularity reduces the efficiency per GPU, while the increased communication overhead, hiders the scalability across GPUs, especially when crossing node boundaries. On the contrary, DP has both higher computational granularity and lower communication volume, allowing for much higher efficiency.

b) DP is memory inefficient as model states are stored redundantly across all data-parallel processes. On the contrary, MP partitions the model states to obtain memory efficiency.

c) Both DP and MP keep all the model states needed over the entire training process, but not everything is required all the time. For example, parameters corresponding to each layer is only needed during the forward propagation and backward propagation of the layer.

Optimizer state partitioning

For a DP degree of N_d, we group the optimizer states into N_d equal partitions, such that the ith data parallel process only updates the optimizer states corresponding to the ith partition. Thus, each data parallel process only needs to store and update $\frac{1}{N_d}$ of the total optimizer states and then only update $\frac{1}{N_d}$ of the parameters. We perform an all-gather across the data parallel process at the end of each training step to get the fully updated parameters across all data parallel process.

Gradient partitioning

As each data parallel process only updates its corresponding parameter partition, it only needs the reduced gradients for the corresponding parameters. Therefore, as each gradient of each layer becomes available during the backward propagation, we only reduce them on the data parallel process responsible for updating the corresponding parameters. After the reduction we no longer need the gradients and their memory can be released. This reduces the memory footprint required to hold the gradients from 2Ψ bytes to $\frac{2\Psi}{N_d}$.

Parameter partitioning

Just as with the optimizer states, and the gradients, each process only stores the parameters corresponding to its partition. When the parameters outside of its partition are required for forward and backward propagation, they are received from the appropriate data parallel process through broadcast. While this may seem to incur significant communication overhead at first glance, we show that this approach only increases the total communication volume of a baseline DP system to 1.5x, while enabling memory reduction proportional to N_d.

Communication volume of standard DP

State-of-art implementation of all-reduce uses a two-step approach, where the first step is a reduce-scatter operation, which reduces different part of the data on different process. The next step is an all-gather operation where each process gathers the reduced data on all the process. The result of these two steps is an all-reduce. Both reduce-scatter and all-gather are implemented using a pipelined approach, that results in a total data movement of Ψ elements (for a data with Ψ elements) for each. Therefore, the standard DP incurs 2Ψ data movement during each training step.

Communication volume of ZeRO-DP

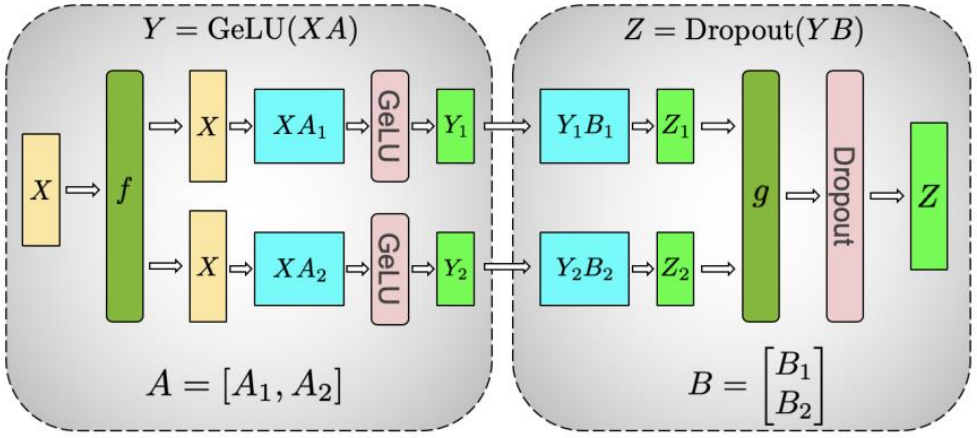
After parameter partitioning, each data parallel process only stores the parameters that it updates. Therefore, during the forward propagation it needs to receives the parameters for all the other partitions. However, this can be pipelined to avoid the memory overhead. Before computing the forward propagation on the part of the model corresponding to a particular partition, the data parallel process responsible for that partition can broadcast the weights to all the data parallel processes. Once the forward propagation for that partition is done, the parameters can be discarded. The total communication volume is thus $\frac{\Psi \times N_d}{N_d} = \Psi$. In other words, we reschedule the parameter all-gather by spreading it across the entire forward propagation, and discarding the parameters once they have been used. Note however that this all-gather needs to happen once again for the backward propagation in the reverse order.

The total communication volume is therefore the sum of the communication volumes incurred by these all-gathers in addition to the communication volume incurred by the reduce-scatter of the gradients. The total volume is therefore 3Ψ which is 1.5x compared to the baseline. Both gradient and parameter partitioning leverage the insight that — not all states of gradients and parameters are needed all the time — to optimize memory by communicating the states judiciously.

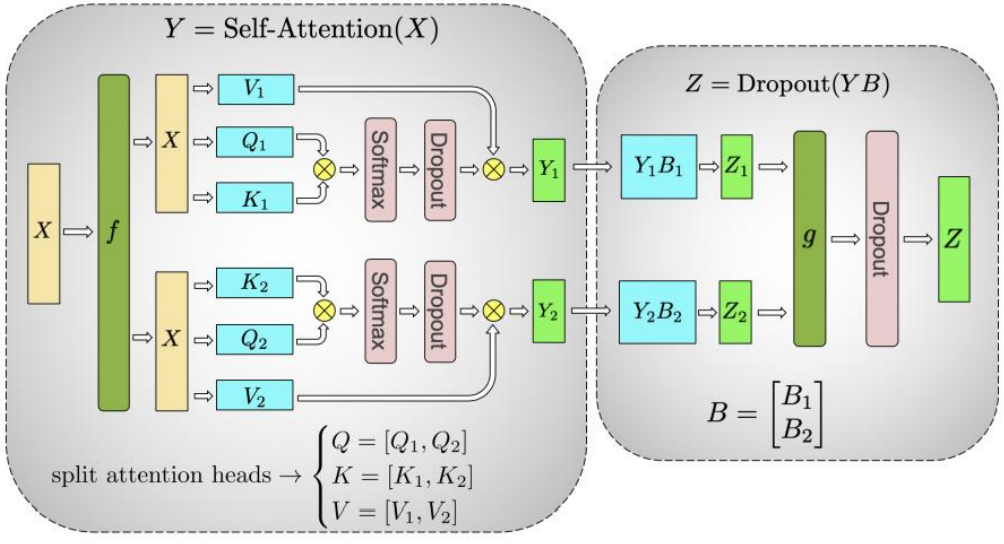
Zero-R

a) MP partitions the model states but often requires replication of the activation memory. For example, if we split the parameters of a linear layer vertically and compute them in parallel across two GPUs, each GPU requires the entire activation to compute its partition

b) For models such as GPT-2 or larger, the arithmetic intensity (ratio of the amount of computation per iteration to amount of activation checkpoints per iteration) is very large ($\geq 10K$) and increases linearly with hidden dimension making it possible to hide the data-movement cost for the activation checkpoints, even when the bandwidth is low.



(a) MLP



(b) Self-Attention

Figure 3. Blocks of Transformer with Model Parallelism. f and g are conjugate. f is an identity operator in the forward pass and all reduce in the backward pass while g is an all reduce in the forward pass and identity in the backward pass.

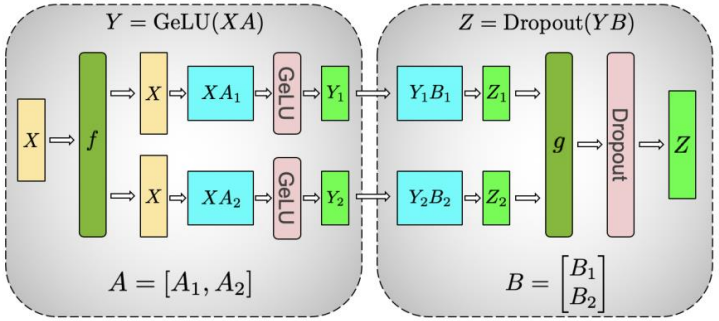
Partitioned activation checkpointing

As discussed in 4.2, MP by design requires a replication of the activations, resulting in redundant copies of the activations across model parallel GPUs. *ZeRO* eliminates this redundancy by partitioning the activations, and only materializes them in a replicated form one activation layer at a time, right before the activation is used in computation. More specifically, once the forward propagation for a layer of a model is computed, the input activations are partitioned across all the model parallel process, until it is needed again during the backprogation. At this point, *ZeRO* uses an all-gather operation to re-materialize a replicated copy of the activations.

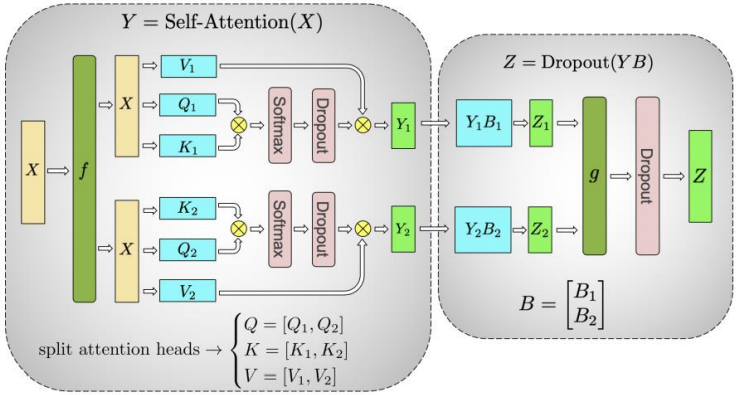
Dealing with Fragmentation in ZeRO-R

- During the forward propagation with activation checkpointing, only selected activations are stored for back propagation while most activations are discarded as they can be recomputed again during the back propagation. This creates an interleaving of short lived memory (discarded activations) and long lived memory (checkpointed activation), leading to memory fragmentation.
- ZeRO does memory defragmentation on-the-fly by pre-allocating contiguous memory chunks for activation checkpoints and gradients, and copying them over to the pre-allocated memory as they are produced.

Communication volume of Megatron



(a) MLP



(b) Self-Attention

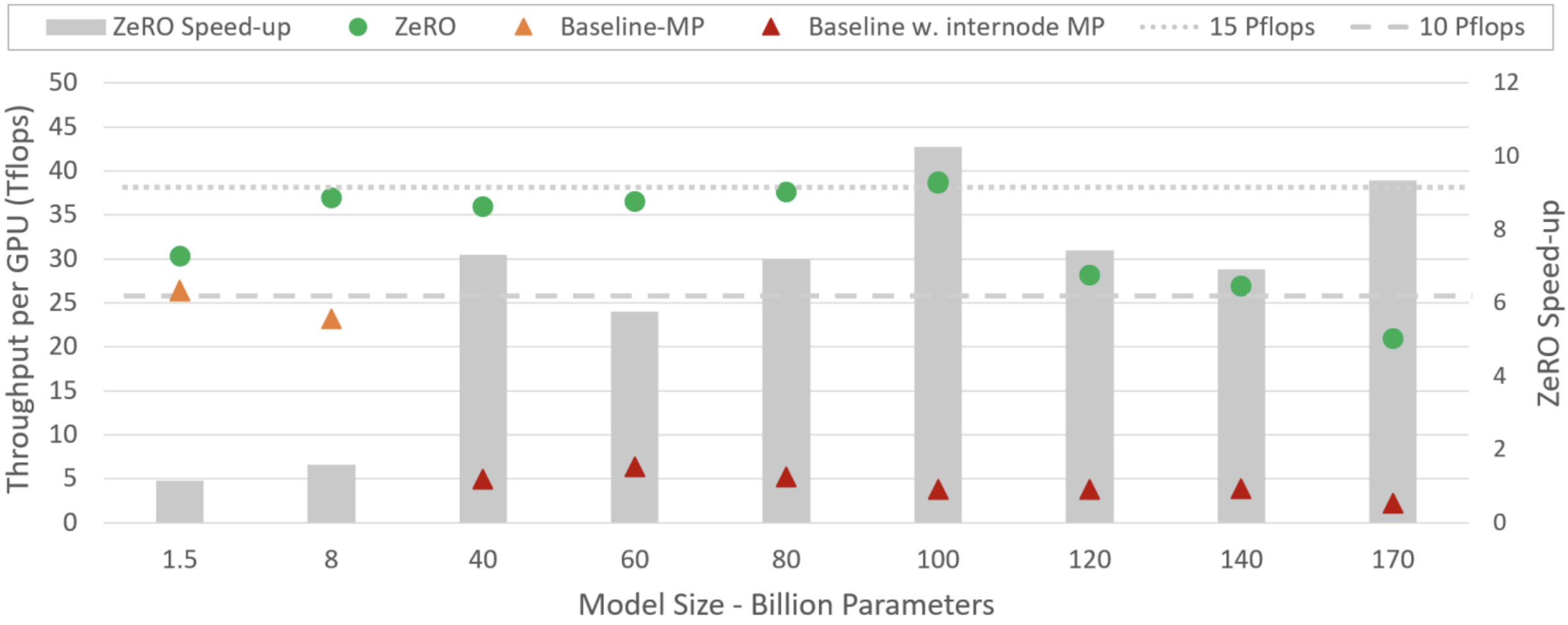
Figure 3. Blocks of Transformer with Model Parallelism. f and g are conjugate. f is an identity operator in the forward pass and all reduce in the backward pass while g is an all reduce in the forward pass and identity in the backward pass.

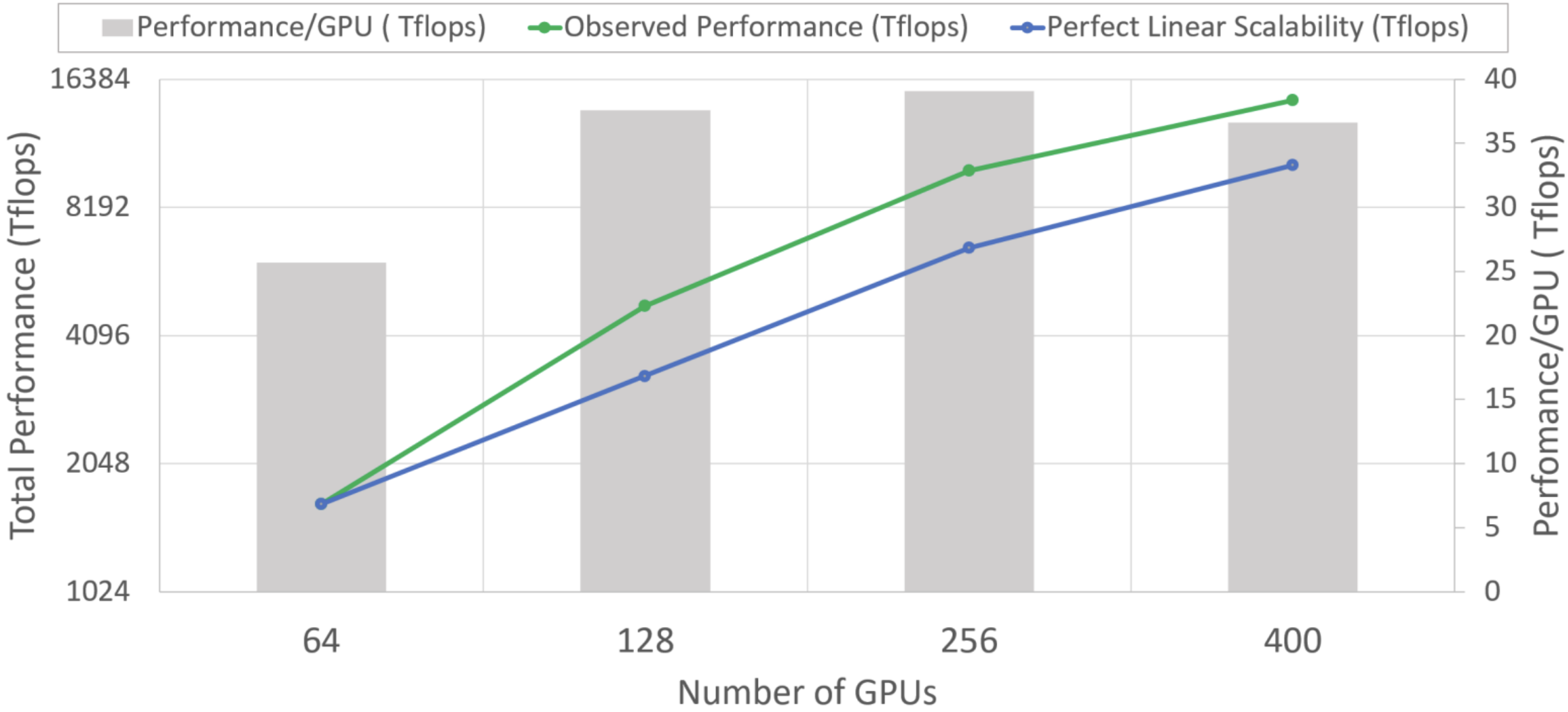
Communication volume of ZeRO-R

- Need extra all-gather per-block for gradient checkpointing

- Less than 10% more compared to what we already need to pay in to to use Megatron-style MP

When MP is used in conjunction with DP, P_a can be used to reduce the data-parallel communication volume by an order of magnitude at the expense of a 10% increase in model-parallel communication volume, and significantly boost efficiency when data-parallel communication is a performance bottleneck. Notice that P_a reduces the activation memory consumption by the MP degree allowing for a proportional increase in batch size. For large models, MP can be as large as 16 (#GPUs on a DGX-2 node), allowing for up to 16x increase in the batch size. The communication volume of a data-parallel training is inversely proportional to the batch size. Therefore, an order of magnitude increase in batch size due to P_a could result in an order-of-magnitude decrease in data-parallel communication volume.


ZeRO Speed-up
ZeRO
Baseline-MP
Baseline w. internode MP
15 Pflops
10 Pflops
Throughput per GPU (Tflops)
ZeRO Speed-up
0
5
10
15
20
25
30
35
40
45
50
0
2
4
6
8
10
12
1.5
8
40
60
80
100
120
140
170
Model Size - Billion Parameters


Performance/GPU (Tflops)
Observed Performance (Tflops)
Perfect Linear Scalability (Tflops)
16384
8192
4096
2048
1024
Total Performance (Tflops)
40
35
30
25
20
15
10
5
0
Perfomance/GPU (Tflops)
64
128
256
400
Number of GPUs

PyTorch FSDP: Experiences on Scaling Fully Sharded Data Parallel

Yanli Zhao
Meta AI
yanlizhao@meta.com

Andrew Gu
Meta AI
andgu@meta.com

Rohan Varma
Meta AI
rvarm1@meta.com

Liang Luo
Meta AI
liangluo@meta.com

Chien-Chin Huang
Meta AI
chienchin@meta.com

Min Xu
Meta AI
m1n@meta.com

Less Wright
Meta AI
less@meta.com

Hamid Shojanazeri
Meta AI
hamidnazeri@meta.com

Myle Ott
Meta AI
myleott@gmail.com

Sam Shleifer
Meta AI
sshleifer@gmail.com

Alban Desmaison
Meta AI
albandes@meta.com

Can Balioglu
Meta AI
balioglu@meta.com

Pritam Damania
Meta AI
pritam.damania@gmail.com

Bernard Nguyen
Meta AI
bernardn@meta.com

Geeta Chauhan
Meta AI
gchauhan@meta.com

Yuchen Hao
Meta AI
haoyc@meta.com

Ajit Mathews
Meta AI
amath@meta.com

Shen Li
Meta AI
shenli@meta.com